

Organisation. Ce TP s'étale sur deux séances (environ 4 h), voire un peu plus.

- La **partie I** ne parle que de probabilités : expériences aléatoires, événements, fréquences. Pas encore de variables aléatoires.
- La **partie II** simule les variables aléatoires discrètes du cours : Bernoulli, binomiale, géométrique, Poisson, lois finies quelconques... et se termine par un problème.
- Les questions étoilées sont facultatives : pour les élèves les plus à l'aise, à garder pour la fin, ou pour les révisions.

Symboles usuellement utilisés :

(♠) : directement exigible au programme (je ne le mets presque jamais en maths... mais en info c'est bon à savoir)

(♡) : je vous conseille de savoir faire cette question

rien : c'est le corps du travail

(*) : plus avancé, ou redondant avec des choses déjà vues

(**) : pour les élèves à la fois les plus intéressés et qui ont du temps

Attention ! L'énoncé d'une question « étoilée » peut être utile pour les questions suivantes et pourtant non-étoilées (c'est la *résolution* des questions étoilées qui porte une étoile, pas la lecture de leur énoncé !)

Règle du jeu sur le langage Python. L'annexe Python du programme officiel n'est pas exhaustive : vous avez le droit d'utiliser les fonctions intégrées (`sum`, `max`, `min`, `abs`, `sorted`, `int`...), les méthodes de listes comme `.count()` ou `.pop(i)`, le test `cle in dico`, etc.

Mais attention : on ne peut les utiliser que si elles ne sont **pas le cœur de la question**. Si l'on vous demande de « compter le nombre d'occurrences d'un objet dans une liste », répondre `liste.count(objet)` sera considéré comme de la triche. En revanche, si ce comptage n'est qu'un ingrédient d'une question plus vaste, `.count()` est le bienvenu.

Le module random

On utilise essentiellement les fonctions suivantes du module `random` (on les retrouve, avec quelques variantes, dans `numpy.random`) :

- `random()` : renvoie un réel pseudo-aléatoire de l'intervalle $[0,1[$, de telle sorte que les nombres obtenus suivent une loi uniforme sur cet intervalle (voir l'exercice 2).
- `randint(a, b)`, où `a` et `b` sont deux entiers : renvoie un entier de $\llbracket a; b \rrbracket$, avec une distribution uniforme. **Attention**, contrairement à `range(a, b)`, la borne `b` peut être atteinte.
- `choice(ma_liste)` : renvoie un élément de `ma_liste` choisi au hasard, de façon équirépartie. Pour un choix non équiréparti, on peut utiliser `numpy.random.choice(ma_liste, p = ...)`, où le paramètre `p` est la liste des probabilités voulues pour les différents éléments de `ma_liste`.
- `shuffle(ma_liste)` : mélange aléatoirement les éléments de `ma_liste`. Attention, cette fonction ne renvoie rien : elle travaille *en place*.

Les modules `random` et `numpy.random` disposent aussi de nombreuses lois non uniformes : `gauss(mu, sigma)`, `expovariate(lambd)`, etc. Je vous renvoie au poly sur les modules et types de données optionnels.

Deux façons d'importer. Dans un énoncé, vous pouvez rencontrer les deux syntaxes suivantes :

```
1 import random as rd          # on écrit ensuite rd.random(), rd.randint(1, 6)...
2 from random import random, randint  # on écrit ensuite random(), randint(1, 6)...
```

La première est la plus propre : on voit toujours de quel module vient chaque fonction, et on évite les collisions de noms (avec la seconde, le nom `random` désigne la *fonction*, plus le module). Il faut néanmoins savoir lire et utiliser les deux.

Dans ce TP, on utilise la première dans la partie I et la seconde dans la partie II.

Justification théorique

Tout ce TP repose sur l'idée suivante : si l'on répète N fois, de façon indépendante, une expérience aléatoire, la *fréquence* F_N de réalisation d'un événement A est proche de sa *probabilité* $p = P(A)$ quand N est grand.

Cela se démontrera en maths, dans les chapitres sur les variables aléatoires discrètes et sur les théorèmes limites. En voici le principe. Notons X_i la variable valant 1 si A est réalisé lors de la i -ème expérience, et 0 sinon. Les X_i sont des variables de Bernoulli de paramètre p , indépendantes.

Notons $F_N = \frac{1}{N} \sum_{i=1}^N X_i$: c'est la fréquence de réalisation de l'événement A sur N réalisations

La *loi faible des grands nombres* donne alors pour tout $\varepsilon > 0$:

$$P(|F_N - p| \geq \varepsilon) \xrightarrow{N \rightarrow +\infty} 0.$$

Autrement dit, quand le nombre d'expériences N est « grand », la fréquence d'apparition de A est *très probablement* une bonne approximation de $p = P(A)$.

NB : On peut montrer que l'écart entre fréquence et probabilité est de l'ordre de $\frac{1}{\sqrt{N}}$. Avec $N = 10\,000$ simulations, n'espérez pas mieux que deux décimales correctes !

I Probabilités

```
import random as rd
```

Exercice 1 *Deux outils : moyenne et fréquence*

1. (♥) Écrire une fonction `moyenne(liste)` qui renvoie la moyenne des termes d'une liste de nombres.
2. (♥) Écrire une fonction `frequence(liste, objet)` qui renvoie la fréquence d'apparition de `objet` dans `liste` :
 - a) avec la méthode `.count()` des listes (voir le poly de Python... ou un moteur de recherche) ;
 - b) sans la méthode `.count()`, « à la main », avec une boucle.

Exercice 2 *Les fonctions `random()` et `randint()` sont-elles uniformes ?*

1. Vérifier expérimentalement que `rd.random()` suit bien une loi uniforme sur $[0, 1[$.
2. Vérifier expérimentalement que `rd.randint(a, b)` suit bien une loi uniforme sur $\llbracket a ; b \rrbracket$.

Protocole. Attention : « loi uniforme » n’a pas le même sens dans les deux questions. Il faut donc suivre des protocoles différents, même si leur structure est la même.

- Pour `randint()` : on choisit des entiers $a \leq b$ et on exécute un *grand nombre de fois* `rd.randint(a, b)`, en comptant le nombre de fois où chaque élément de $\llbracket a; b \rrbracket$ est obtenu. À la fin, on vérifie que les *fréquences* obtenues sont toutes proches de $\frac{1}{b-a+1}$.

- Pour `random()` : `rd.random()` produit des réels, et la probabilité d’obtenir un réel donné est nulle. On s’intéresse donc plutôt à des intervalles. On choisit des réels $x \leq y$ dans $[0, 1]$, on exécute un grand nombre de fois `rd.random()`, et on vérifie que la fréquence de l’événement « $x \leq \text{rd.random()} \leq y$ » est proche de $y - x$.

`rd.random()` simule en fait une variable aléatoire « continue » (on dira à *densité*), notion qui sera étudiée plus tard en maths. Pour l’instant, reprenez sa **propriété fondamentale** :

pour tous réels $0 \leq x \leq y \leq 1$, la probabilité que `rd.random()` tombe dans l’intervalle $[x, y]$ (ou $[x, y[$, ou $]x, y]$...) est égale à sa longueur $y - x$.

Exercice 3 *Fabriquer `randint()` et `choice()`, lancer des dés*

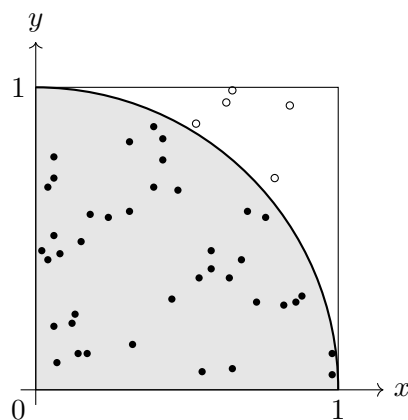
1. À l’aide de la fonction `rd.random()`, écrire une fonction `mon_randint(a, b)` qui fait la même chose que `rd.randint(a, b)`.
2. À l’aide de la fonction `rd.randint()`, écrire une fonction `mon_choice(maliste)` qui fait la même chose que `rd.choice(maliste)`.
3. Simuler le lancer d’un dé équilibré à n faces.
4. Simuler l’expérience aléatoire consistant à lancer deux dés équilibrés à 6 faces et à additionner leurs résultats.

Vérifier que les fréquences des différentes sommes sont conformes aux probabilités prévues par la théorie.

Exercice 4 *Approximation de π par une méthode de Monte-Carlo*

D’une manière générale, les méthodes de Monte-Carlo sont une famille d’algorithmes visant à calculer une valeur approchée en simulant des processus aléatoires. Elles sont notamment utilisées pour calculer des aires ou des volumes, en physique statistique, en finance... et elles ont servi à mettre au point la bombe atomique. Voir par exemple Wikipédia (cliquer ici).

Ici, rien d’aussi spectaculaire : on choisit « au hasard, et uniformément » des points dans le carré $[0, 1]^2$. Quelle est la probabilité qu’ils tombent dans le quart de disque de centre $(0, 0)$ et de rayon 1 ?



Écrire une fonction `pi_monte_carlo(N)` qui tire N points au hasard dans le carré et renvoie une valeur approchée de π .

Exercice 5 *Pile ou face*

On considère une pièce ayant une probabilité $p \in]0, 1[$ de donner Pile.

1. Écrire une fonction `lancers(n, p)` qui simule n lancers (indépendants) de cette pièce, et qui renvoie une liste de n nombres valant 0 ou 1 : les 0 représentent les Face et les 1 représentent les Pile.
2. Écrire une fonction `nb_piles(n, p)` qui renvoie le nombre de Pile obtenus à l'issue de n lancers.
3. a) Écrire une fonction `estime_au_moins_un_pile(n, p, nb_essais=10000)` qui estime la probabilité d'obtenir au moins un Pile au cours de n lancers.
b) Calculer cette probabilité théoriquement, et comparer pour plusieurs valeurs de n et p .

Exercice 6 *Appelez la SPA !*

Ariane et Boualem font un pari. Ariane lance des fléchettes : à chaque lancer, elle a une chance sur deux de toucher la cible (un chat). La partie continue tant que le nombre de succès d'Ariane ne dépasse pas son nombre d'échecs de 2 unités (ou l'inverse) : dans le premier cas, Ariane gagne ; dans le second, Boualem gagne.

1. Simuler cette aventure, à l'aide d'une fonction `appelez_la_spa()` qui :
 - affiche le déroulé des opérations (et en particulier le nom du vainqueur) ;
 - renvoie le nombre total de lancers.
2. On généralise : Ariane touche la cible avec une probabilité $p \in]0, 1[$ à chaque lancer, les lancers étant indépendants. Estimer la probabilité qu'Ariane gagne, pour $p = 0,5$, $p = 0,6$ et $p = 0,8$.
3. (*) Montrer que la probabilité qu'Ariane gagne vaut $\frac{p^2}{p^2 + (1-p)^2}$, et comparer avec la question précédente.
Indication : La partie ne peut s'arrêter qu'après un nombre pair de lancers. Regrouper les lancers deux par deux.
4. (*) **Durée de la partie.** Soit $k \in \mathbb{N}^*$. Estimer la probabilité que la partie dure strictement plus de $2k$ lancers, et comparer avec $(2p(1-p))^k$. Démontrer ce résultat.
5. **Question inverse.** Quelle valeur de p faut-il pour qu'Ariane gagne avec probabilité 0,9 ?
 - a) Répondre par le calcul, à l'aide de la question 3.
 - b) Répondre à l'aide d'une fonction `dichotomie(cible, a=0.5, b=1, epsilon=1e-6)`, qui renvoie une valeur approchée à `epsilon` près du réel $p \in [a, b]$ tel que $\frac{p^2}{p^2 + (1-p)^2} = \text{cible}$. Et pour une probabilité de 0,99 ?
6. (**) **Partie à écart m .** On change la règle : la partie s'arrête dès que l'écart entre les succès et les échecs d'Ariane atteint $m \in \mathbb{N}^*$ (l'énoncé initial correspond à $m = 2$).
 - a) Écrire une fonction `spa_ecart(p=0.5, m=2)` qui simule une partie (sans affichage) et renvoie `True` si Ariane gagne.
 - b) Estimer la probabilité qu'Ariane gagne pour $p = 0,6$ et $m \in \llbracket 1 ; 4 \rrbracket$. En vous inspirant de la formule de la question 3, conjecturer une formule pour cette probabilité en fonction de p et m .

Exercice 7 (**) *Le tirage au sort des cadeaux*

Pour les fêtes de fin d'année, $n \geq 2$ personnes s'organisent ainsi : chacune écrit son nom sur un papier, on met les n papiers dans un chapeau, puis chaque personne tire à son tour, au hasard et sans remise, un papier. Elle devra offrir un cadeau à la personne dont elle a tiré le nom. Le tirage est *réussi* si personne n'a tiré son propre nom (sinon, il faut recommencer).

On note p_n la probabilité que le tirage soit réussi. En Python, on numérote les personnes et les noms de 0 à $n - 1$.

1. Écrire une fonction `tirage(n)` qui simule le tirage au sort et renvoie une liste `t` de longueur n , où `t[i]` est le numéro du nom tiré par la personne numéro i .
2. Écrire une fonction `est_reussi(t)` qui renvoie `True` si le tirage `t` est réussi, et `False` sinon.
3. Écrire une fonction `estime_proba_reussite(n, nb_essais=10000)` qui estime p_n . Afficher les estimations de p_n pour $n \in \llbracket 2; 12 \rrbracket$. Que constate-t-on ? Comparer avec `exp(-1)` (module `math`).
4. (**) **Calcul de p_n .** On note d_n le nombre de façons d'attribuer les n noms aux n personnes (une personne recevant un et un seul nom) de sorte que personne ne reçoive son propre nom. En mathématiques, on numérote les personnes et les noms de 1 à n .
 - a) Justifier que $p_n = \frac{d_n}{n!}$, puis calculer d_1 (avec la même définition, pour une personne), d_2 et d_3 .
 - b) Soit $n \geq 3$. Montrer que $d_n = (n - 1)(d_{n-1} + d_{n-2})$.

Indication : Notons k le nom tiré par la personne n . Distinguer deux cas, selon que la personne k a tiré le nom n ou non. Dans le second cas, on pourra « faire sortir la personne n du jeu » en donnant le nom k à la personne qui a tiré le nom n .
 - c) En déduire que, pour tout $n \geq 3$, $p_n - p_{n-1} = -\frac{1}{n}(p_{n-1} - p_{n-2})$, puis que pour tout $n \geq 2$, $p_n - p_{n-1} = \frac{(-1)^n}{n!}$.
 - d) En déduire que $p_n = \sum_{k=0}^n \frac{(-1)^k}{k!}$, puis la limite de p_n quand n tend vers $+\infty$.
 - e) Écrire une fonction `proba_exacte(n)` qui calcule p_n , et comparer avec les estimations de la question 3.

Exercice 8 (**) *Démasquer les tricheurs : les séries de Pile*

Demandez à quelqu'un d'écrire une suite de 200 résultats de Pile ou Face qui ait l'air aléatoire : il évitera presque toujours les longues séries de Pile consécutifs, qui lui sembleront « trop peu aléatoires ». Le but de cet exercice est de montrer qu'il a tort, et qu'au contraire, l'absence de longues séries est un bon indice de tricherie.

On reprend les notations de l'exercice 5 : on lance n fois une pièce donnant Pile avec probabilité $p \in]0, 1[$, les lancers étant indépendants, et on note $q = 1 - p$. Pour $k \in \mathbb{N}^*$, une *série de k Pile* est une suite de k lancers consécutifs donnant tous Pile.

1. Écrire une fonction `plus_longue_serie(resultats)` qui prend en entrée une liste de 0 et de 1 (comme celles renvoyées par la fonction `lancers` de l'exercice 5), et qui renvoie la longueur de la plus longue suite de 1 consécutifs.

Par exemple, `plus_longue_serie([1, 1, 0, 1, 1, 1, 0, 0, 1])` doit renvoyer 3.
2. Écrire une fonction `estime_proba_serie(n, k, p=0.5, nb_essais=10000)` qui estime la probabilité d'obtenir au moins une série de k Pile au cours de n lancers. L'utiliser pour $n = 200$, $p = \frac{1}{2}$ et $k \in \llbracket 4; 10 \rrbracket$.
3. Une personne affirme avoir lancé 200 fois une pièce équilibrée, et n'avoir jamais obtenu plus de 4 Pile d'affilée. La croyez-vous ?

4. (**) **Calcul exact.** On fixe $k \in \mathbb{N}^*$ et, pour tout $n \in \mathbb{N}$, on note A_n l'événement « au cours des n premiers lancers, on obtient au moins une série de k Pile », et $u_n = P(\overline{A_n})$.
- Que vaut u_n pour $n < k$? Calculer u_k .
 - Soit $n \geq k$. Pour $j \in \llbracket 1; k \rrbracket$, on note B_j l'événement « les $j - 1$ premiers lancers donnent Pile et le j -ième donne Face », et B_{k+1} l'événement « les k premiers lancers donnent Pile ».
 — Justifier que $(B_1, \dots, B_{k+1})$ est un système complet d'événements.
 — Montrer que $P(B_{k+1} \cap \overline{A_n}) = 0$ et que, pour tout $j \in \llbracket 1; k \rrbracket$, $P(B_j \cap \overline{A_n}) = p^{j-1}q u_{n-j}$.
Indication : Si B_j est réalisé avec $j \leq k$, une série de k Pile ne peut se trouver que parmi les lancers numéros $j + 1$ à n .
 - En déduire que pour tout $n \geq k$, $u_n = \sum_{j=1}^k p^{j-1}q u_{n-j}$.
 - Écrire une fonction `proba_serie(n, k, p=0.5)` qui renvoie la valeur exacte de $P(A_n)$, et la comparer aux estimations de la question 2.

II Variables aléatoires discrètes

Pour varier les plaisirs, on commence le fichier de cette deuxième partie avec la ligne :

```
from random import randint, random, choice
```

Simuler une variable aléatoire : de quoi parle-t-on ? D'une manière générale, « simuler une variable aléatoire X » consiste à écrire une fonction `simule_X()` qui renvoie un élément de $X(\Omega)$, de telle sorte que pour tout $v \in X(\Omega)$, la probabilité que `simule_X()` renvoie v soit égale à $P(X = v)$. Pour « vérifier expérimentalement les résultats théoriques sur la loi de X », on lance la fonction `simule_X()` un grand nombre de fois, on calcule la fréquence de chaque valeur obtenue, et on vérifie que ces fréquences sont proches des probabilités théoriques données par la loi de X .

Attention ! Pour *simuler* X , on ne veut pas utiliser sa loi théorique, mais rester au plus près de la *définition* de X . Par exemple, pour simuler une variable binomiale, on ne veut pas voir de $\binom{n}{k} p^k (1-p)^{n-k}$ dans le code, mais réellement simuler n épreuves de Bernoulli et compter les succès. Fondamentalement, notre but est de fabriquer à l'aide du module `random` une *machine qui se comporte du point de vue probabiliste de la même manière que le phénomène qu'on cherche à simuler*, et NON calculer en python les probas données par la théorie.

Vocabulaire : Moyenne, Variance et Fonction de Répartition empiriques

Soit X une variable aléatoire discrète admettant une espérance et variance. Soient $X_1, \dots, X_N$ des variables aléatoires indépendantes de même loi que X (en pratique ici : les résultats de N simulations de X). On définit :

- la *moyenne empirique* $M_N = \frac{1}{N} \sum_{i=1}^N X_i$. La *loi faible des grands nombres* (vue plus tard dans l'année, et qu'on a déjà utilisée implicitement dans la **Partie I**) donne que pour N « grand », M_N donne une *estimation* de $E(X)$
- la *variance empirique* $\frac{1}{N} \sum_{i=1}^N (X_i - M_N)^2$ qui donne de même une estimation de $V(X)$.
- pour tout réel t , la fréquence de l'événement $(X_k \leq t)$ au cours des N simulations, appelée *fonction de répartition empirique* et notée $F_N(t)$, estime $F_X(t) = P(X \leq t)$ (c'est exactement la situation de la partie I, avec l'événement $(X \leq t)$).

« **Rappels** » de tracé (ou cf. le poly Annexe III du cours de Python, et le TP correspondant !). On utilise le module `matplotlib`, qu'on importe par `import matplotlib.pyplot as plt`. Rappels :

- `plt.plot(liste_x, liste_y)` crée la ligne brisée reliant les points dont les abscisses et les ordonnées sont données respectivement par les listes `liste_x` et `liste_y`. Paramètres optionnels utiles : `color='red'`, `linestyle='--'`, `marker='o'` (pour marquer les points), `label='...'` (pour la légende).
- `plt.bar(liste_x, liste_hauteurs, width=0.5)` crée un diagramme en bâtons : une barre de largeur 0,5 centrée en chaque abscisse, de hauteur donnée. C'est l'outil idéal pour représenter des fréquences.
- `plt.step(liste_x, liste_y, where='post')` crée une fonction en escalier, constante entre deux abscisses consécutives : idéal pour une fonction de répartition.
- `plt.legend()` crée la légende ;
- `plt.show()` affiche sur la même figure tous les éléments définis par les commandes précédentes

Exercice 9 Variables de Bernoulli, variables binomiales

1. (♠) Écrire une fonction `bernoulli(p)` qui simule une variable de Bernoulli de paramètre p , puis vérifier expérimentalement le cours sur la loi de Bernoulli.
2. (♠) Écrire une fonction `variable_binomiale(n, p)` qui simule une variable de loi binomiale $\mathcal{B}(n, p)$, à l'aide de la fonction précédente.
3. On veut vérifier le cours sur la loi binomiale.
 - a) Écrire une fonction `k_parmi_n(k, n)` qui renvoie $\binom{n}{k}$.
 - b) Écrire une fonction `proba_binomiale(n, p, k)` qui renvoie la valeur vue en cours de $P(X = k)$, où X suit une loi $\mathcal{B}(n, p)$.
 - c) Écrire enfin un script qui compare, pour $k \in \llbracket 0; n \rrbracket$, la valeur théorique de $P(X = k)$ et la fréquence du résultat k parmi un grand nombre de résultats renvoyés par la fonction de la question 2.
4. (*)
 - a) Écrire une fonction `variance_empirique(liste)`.
 - b) Comparer la moyenne et la variance empiriques d'un grand nombre de simulations de `variable_binomiale(n, p)` avec les valeurs théoriques de l'espérance et de la variance.

Exercice 10 Simuler une loi géométrique

1. (♠) Écrire une fonction `variable_geometrique(p)` qui simule une variable X de loi géométrique $\mathcal{G}(p)$, à l'aide de la fonction `bernoulli` de l'exercice précédent.
2. (♠) Estimer l'espérance et la variance de X par la moyenne et la variance empiriques d'un grand nombre de simulations, et comparer avec les résultats théoriques vus en cours, pour plusieurs valeurs de p .
3. On vérifie maintenant que la loi théorique de X correspond raisonnablement aux fréquences observées.
 - a) **À l'aide d'un dictionnaire.** Écrire une fonction `estime_loi(p, nb_essais=10000)` qui simule `nb_essais` fois une variable de loi géométrique de paramètre p , et qui renvoie un dictionnaire dont les clés sont les valeurs obtenues, et les valeurs les fréquences correspondantes.
Vérifier que les fréquences sont raisonnablement compatibles avec les probabilités théoriques.
 - b) **Graphiquement.** Écrire une fonction `trace_loi_geometrique(p, nb_essais=10000)` qui représente sur une même figure :
 - les fréquences observées, sous forme de diagramme en bâtons (on pourra utiliser le dictionnaire précédent) ;
 - les probabilités théoriques $P(X = k)$, sous forme de points reliés.
4. **Fonction de répartition empirique.**
 - a) Écrire une fonction `repartition_empirique(liste, t)` qui renvoie la proportion des éléments de `liste` inférieurs ou égaux à `t`.
 - b) Calculer, pour tout $k \in \mathbb{N}$, $F_X(k) = P(X \leq k)$.
 - c) Écrire une fonction `trace_repartition_geometrique(p, nb_essais=10000, k_max=15)` qui représente sur une même figure la fonction de répartition empirique d'un échantillon de `nb_essais` simulations, et les valeurs théoriques $F_X(k)$, pour $k \in \llbracket 0; k_{\max} \rrbracket$.
5. (*) **Absence de mémoire.** Soit $n, k \in \mathbb{N}$. Estimer la probabilité conditionnelle $P_{(X > n)}(X > n + k)$ et la comparer à $P(X > k)$. Commenter.

Exercice 11 *Tirages sans remise*

Une urne contient a boules blanches et b boules noires. On tire successivement et sans remise n boules de cette urne ($n \leq a + b$), et on note X le nombre de boules blanches obtenues.

1. Écrire une fonction `tirages_sans_remise(a, b, n)` qui simule la variable X . On pourra proposer deux versions :
 - a) l'une qui tient à jour les nombres de boules blanches et noires restant dans l'urne ;
 - b) l'autre qui représente l'urne par une liste, dont on retire les boules une à une.
2. On admet (ou on se rappelle...) que pour tout $k \in \llbracket 0; n \rrbracket$, $P(X = k) = \frac{\binom{a}{k} \binom{b}{n-k}}{\binom{a+b}{n}}$, et que $E(X) = \frac{na}{a+b}$. Le vérifier expérimentalement.

Exercice 12 *Simuler une variable aléatoire finie quelconque*

1. (♠) On considère une variable aléatoire X telle que $X(\Omega) = \llbracket 0; n \rrbracket$, de loi connue : on note $p_k = P(X = k)$ pour tout $k \in \llbracket 0; n \rrbracket$.
Écrire une fonction `simule_X(liste_probas)` qui prend en entrée la liste $[p_0, p_1, \dots, p_n]$, et renvoie un entier compris entre 0 et n suivant la loi de X .
Indication : Découper l'intervalle $[0, 1[$ en $n + 1$ intervalles de longueurs $p_0, p_1, \dots, p_n$.
2. Vérifier expérimentalement votre fonction avec la loi $p_0 = 0,1$, $p_1 = 0,6$, $p_2 = 0,15$, $p_3 = 0,15$.
3. Simuler une variable aléatoire finie à **valeurs quelconques**, connaissant sa loi.
4. (*) Qu'affiche le script suivant ?

```
1 S = 0
2 for k in range(10):
3     S += 0.1
4 print(S, S < 1)
```

Quel problème cela peut-il poser pour votre fonction `simule_X`, avec `liste_probas = [0.1] * 10` ? Proposer une version qui en soit protégée.

Exercice 13 *Simuler une loi de Poisson*

Dans cet exercice, a désigne un réel strictement positif, et X une variable de loi de Poisson $\mathcal{P}(a)$.

Attention : on ne peut pas appeler une variable `lambda` en Python : c'est un mot-clé (hors programme) qui sert à définir des fonctions de manière abrégée. D'où le choix de noter a le paramètre.

1. (♠) Simuler X en utilisant la même idée que dans l'exercice précédent (inversion de la fonction de répartition) :
 - a) dans une première version, `simule_poisson_naif(a)`, en calculant les probabilités $P(X = k)$ à l'aide de la fonction `factorial` du module `math` ;
 - b) (*) dans une seconde version, `simule_poisson(a)`, sans aucun calcul de factorielle ni de puissance.
2. Vérifier expérimentalement la loi de X , ainsi que son espérance et sa variance.
3. (♠) **Approximation par une loi binomiale.** On admet (cela sera vu plus tard dans le cours de maths) que pour n grand, une variable de loi $\mathcal{B}\left(n, \frac{a}{n}\right)$ suit approximativement une loi $\mathcal{P}(a)$.
Écrire une fonction `simule_poisson_binomiale(a, n=1000)` qui simule une variable de loi $\mathcal{B}\left(n, \frac{a}{n}\right)$, et comparer sa loi empirique avec la loi $\mathcal{P}(a)$.

Problème 14 *Chi-Fou-Mi*

Nous allons simuler des parties de Chi-Fou-Mi, également appelé de manière plus claire « Papier-Caillou-Ciseau ».

Rappelons les règles. À chaque manche, les deux adversaires jouent simultanément, au choix, *Papier*, *Caillou* ou *Ciseau*, avec les règles suivantes :

- « Papier » bat « Caillou », qui bat « Ciseau », qui bat « Papier » ;
- si les deux adversaires jouent la même chose, la manche est nulle.

Une partie à écart m (où $m \in \mathbb{N}^*$) se termine dès que l'un des deux adversaires a gagné m manches de plus que l'autre : c'est le vainqueur. Généralement, on prend $m = 2$.

On suppose que les deux adversaires jouent au hasard : à chaque manche, chacun choisit l'une des trois attaques de façon équiprobable, indépendamment de l'autre et des manches précédentes. On note S_m le nombre de manches (y compris les manches nulles) que dure une partie à écart m .

Le but du problème est d'écrire un programme qui simule des parties de Chi-Fou-Mi, puis d'étudier la loi et l'espérance de S_m .

1. Réfléchissez à la manière de stocker les informations capitales. Voici une idée de variables globales que vous pouvez utiliser (sans obligation) pour fluidifier votre code :

```
1  attaques = ["Papier", "Caillou", "Ciseau"]
2  victime = {"Papier": "Caillou", "Caillou": "Ciseau", "Ciseau": "Papier"}
```

Comprenez-vous comment on peut utiliser conjointement ces deux variables ?

2. Écrivez en français la structure du code nécessaire pour accomplir la tâche souhaitée : une seule ou plusieurs fonctions ? On précisera leurs paramètres et leur utilité, et les structures internes nécessaires (boucles, conditions), sans entrer forcément dans les détails de la syntaxe Python.

Page suivante, vous aurez une proposition de structure, mais réfléchissez-y par vous-même d'abord.

Une proposition de structure pour le programme :

- les importations nécessaires ;
- les variables `attaques` et `victime` décrites plus haut ;
- une fonction `resultat(attaque_0, attaque_1)` qui prend deux attaques en entrée (une attaque étant représentée par une chaîne de caractères : "Papier", "Caillou" ou "Ciseau") et renvoie `-1` si la manche est nulle, et le numéro de l'attaque gagnante (0 ou 1) sinon ;
- une fonction `chi_fou_mi(m=2, affichage=False)` qui :
 - simule une partie à écart m ;
 - renvoie le nombre de manches de la partie (y compris les manches nulles) ;
 - si le booléen `affichage` vaut `True`, affiche le déroulement de la partie (le choix du nom des adversaires est laissé à votre appréciation, et peut être l'occasion d'ajouter un paramètre à la fonction).

Exemples d'exécution, en laissant à m sa valeur par défaut 2 :

- avec `affichage` valant sa valeur par défaut `False` :

```
>>> chi_fou_mi()
3
```

- avec `affichage` valant `True` :

```
>>> chi_fou_mi(affichage=True)
manche 1
Ariane : Ciseau !!
Boualem : Papier !!
un point pour Ariane
scores :
Ariane : 1
Boualem : 0
-----
manche 2
Ariane : Papier !!
Boualem : Ciseau !!
un point pour Boualem
scores :
Ariane : 1
Boualem : 1
-----
```

```
manche 3
Ariane : Ciseau !!
Boualem : Caillou !!
un point pour Boualem
scores :
Ariane : 1
Boualem : 2
-----
manche 4
Ariane : Ciseau !!
Boualem : Caillou !!
un point pour Boualem
scores :
Ariane : 1
Boualem : 3
-----
Le vainqueur est : Boualem
4
```

- une fonction `estime_loi_chifoumi(m=2, nb_essais=100000)` qui renvoie un dictionnaire dont les clés sont les valeurs obtenues par la fonction précédente, et les valeurs correspondantes leurs fréquences ;
- une fonction `estime_esperance_chifoumi(m=2, nb_essais=100000)` qui estime $E(S_m)$.

3. Programmer le tout (en suivant cette structure, ou la vôtre).

It's conjecture time! Les trois questions suivantes sont de difficulté croissante. Allez aussi loin que vous le pouvez : le niveau 1 est pour tout le monde, le niveau 3 pour les plus courageux/ses.

4. Niveau 1 : le cas $m = 1$.

- a) Montrer que S_1 suit une loi usuelle, dont on précisera le ou les paramètres.
- b) Vérifier expérimentalement la cohérence de vos simulations avec la théorie (loi et espérance).

5. Niveau 2 : une conjecture pour $E(S_m)$.

- a) Estimer $E(S_m)$ pour $m \in \llbracket 1 ; 8 \rrbracket$, et représenter graphiquement ces estimations en fonction de m .
- b) Conjecturer une formule pour $E(S_m)$ en fonction de m .
Indication : Une croissance plus rapide que linéaire ? Calculer les quotients des estimations par m^2 .

6. () Niveau 3 : la loi de S_2 .** Dans cette question, $m = 2$. Pour tout $n \in \mathbb{N}$, on note :

- Z_n l'événement « à l'issue de n manches, la partie n'est pas terminée et les scores sont égaux », et $a_n = P(Z_n)$;
- U_n l'événement « à l'issue de n manches, la partie n'est pas terminée et l'un des adversaires a un point d'avance », et $b_n = P(U_n)$.

a) Donner a_0, b_0, a_1 et b_1 .

b) Montrer que pour tout $n \in \mathbb{N}$, $a_{n+1} = \frac{a_n + b_n}{3}$ et $b_{n+1} = \frac{2a_n + b_n}{3}$.

c) En déduire que pour tout $n \in \mathbb{N}$, $b_{n+2} = \frac{2}{3}b_{n+1} + \frac{1}{9}b_n$, puis exprimer b_n en fonction de n .

d) Montrer que pour tout $n \in \mathbb{N}^*$, $P(S_2 = n) = \frac{b_{n-1}}{3}$. Vérifier que $\sum_{n=1}^{+\infty} P(S_2 = n) = 1$, et interpréter.

e) Calculer $E(S_2)$. Est-ce cohérent avec le niveau 2 ?

f) Comparer la loi obtenue avec les estimations données par `estime_loi_chifoumi(2)`.

Exercice 15 (**) *Le problème des partis*

Attention ! Utilise de la récursivité

Cet exercice utilise la *récursivité* (une fonction qui s'appelle elle-même) dans les questions **3** à **5**. Vous l'avez vue l'an dernier, et nous la reverrons plus tard dans l'année. Les questions **1**, **2** et **6** peuvent être traitées sans.

- Si vous êtes à l'aise avec la récursivité, vous pouvez traiter cet exercice dès maintenant.
- Sinon, gardez-le pour vos révisions, après notre TP sur la récursivité : c'est une très bonne application.

Pour mémoire, une fonction récursive doit comporter :

- un ou plusieurs *cas de base*, traités directement, sans appel récursif ;
- des appels récursifs portant sur des valeurs « plus proches » des cas de base, pour être sûr que les appels s'arrêtent.

Ici, la relation de la question **2b** se traduit presque mot pour mot en une fonction récursive, avec pour cas de base ceux de la question **2a**.

En 1654, le chevalier de Méré pose à Pascal le problème suivant. Deux joueurs, A et B, misent la même somme et jouent une suite de manches ; le premier qui gagne N manches remporte la totalité des mises. Mais la partie est interrompue avant la fin, alors qu'il manque a manches à A et b manches à B pour gagner. Comment partager équitablement les mises ? La correspondance entre Pascal et Fermat sur ce problème est considérée comme l'acte de naissance du calcul des probabilités.

La réponse de Pascal et Fermat est de partager les mises proportionnellement aux probabilités qu'avait chaque joueur de gagner, si la partie avait continué. On suppose que les manches sont

indépendantes, et que A gagne chaque manche avec une probabilité $p \in]0, 1[$; on note $q = 1 - p$. Pour $a, b \in \mathbb{N}$, non tous deux nuls, on note $\pi(a, b)$ la probabilité que A gagne, sachant qu'il lui manque a manches et qu'il en manque b à B.

1. a) Écrire une fonction `fin_de_partie(a, b, p=0.5)` qui simule la fin de la partie, et renvoie `True` si A gagne.
 b) Écrire une fonction `estime_partis(a, b, p=0.5, nb_essais=10000)` qui estime $\pi(a, b)$.
 c) L'exemple historique : avec $p = \frac{1}{2}$, il manque 1 manche à A et 2 manches à B. Comment partager les mises ?
2. a) Que valent $\pi(0, b)$ pour $b \geq 1$, et $\pi(a, 0)$ pour $a \geq 1$?
 b) Montrer que pour tous $a, b \in \mathbb{N}^*$, $\pi(a, b) = p \pi(a - 1, b) + q \pi(a, b - 1)$.
3. **Récursivité.** En déduire une fonction **récursive** `partis(a, b, p=0.5)` qui renvoie $\pi(a, b)$. La comparer aux estimations de la question 1.
4. (*) **Coût.**
 a) Écrire une fonction récursive `nb_appels(a, b)` qui renvoie le nombre total d'appels à la fonction `partis` provoqués par l'appel `partis(a, b)` (y compris celui-ci).
 b) Calculer `nb_appels(n, n)` pour $n \in \{5, 10, 12, 14\}$, et mesurer le temps de calcul de `partis(n, n)`. Commenter.
5. (*) **Mémoïsation.** Pour éviter de recalculer plusieurs fois les mêmes valeurs, on les stocke dans un dictionnaire au fur et à mesure qu'on les calcule. Écrire une fonction `partis_memo(a, b, p=0.5, memo={})` qui fait cela, et calculer `partis_memo(100, 100)`.
6. (**) **Formule de Pascal.** On imagine qu'on joue, quoi qu'il arrive, exactement $a + b - 1$ manches (les manches jouées après la fin réelle de la partie ne comptant pas).
 a) Montrer que A gagne la partie si et seulement si il gagne au moins a de ces $a + b - 1$ manches.
 b) On note Y le nombre de manches gagnées par A parmi ces $a + b - 1$ manches. Reconnaître la loi de Y , et en déduire que $\pi(a, b) = P(Y \geq a) = \sum_{k=a}^{a+b-1} \binom{a+b-1}{k} p^k q^{a+b-1-k}$.
 c) Programmer cette formule à l'aide de la fonction `proba_binomiale` de l'exercice 9, et la comparer à la fonction `partis`.